

# Interview Script: Walking Through the FollowUpHub Backend

---

If an interviewer asks, "Walk me through your backend codebase," this is exactly how you should structure your response. The key is to start high-level, dive into the entry points, trace a standard request, and then highlight your most complex features (Cron + WebSockets).

Do not read line-by-line. Instead, explain the *architecture* and *intent* behind the code.

---

## 1. The High-Level Overview (Start Here)

---

*Keep this under 60 seconds to set the stage.*

### **You say:**

"I built the backend as a standalone Node.js and Express API, entirely decoupled from the Next.js frontend. I chose this decoupled architecture specifically because the application relies heavily on background cron jobs for the Reminder Engine, and stateful WebSockets via Socket.IO for real-time notifications. These are things that serverless functions struggle with, so a dedicated Express server was the right tool for the job. For data, I'm using PostgreSQL managed by Drizzle ORM, and I use Redis for rate-limiting and Socket.IO multi-process scaling."

---

## 2. The Entry Points (`server.js` & `app.js`)

---

*Show them where the app boots up.*

**You say:**

"If we start at the entry point in `server.js`, you'll see I separated the server initialization from the Express app configuration. `app.js` is strictly where I define my middleware chain and mount my routes.

Back in `server.js`, this file acts as the orchestrator. It spins up the HTTP server, connects to Redis, binds Socket.IO to the server, and starts the background cron jobs.

*[Optional: Point to Graceful Shutdown]* One thing I want to highlight here is the graceful shutdown logic I implemented at the bottom of `server.js`. If the server receives a `SIGTERM` from the hosting provider during a deployment, it catches it, stops accepting new HTTP connections, pauses the cron jobs, and cleanly shuts down the database and Redis connections before exiting. This prevents in-flight requests from being dropped and data from corrupting."

---

## 3. Data Layer (`db/schema.js`)

---

*Show them how you model data.*

**You say:**

"Moving into the `db` directory, I used Drizzle ORM over something heavier like Prisma. I chose Drizzle because it's incredibly lightweight, has zero cold-start issues, and the syntax maps very closely to raw SQL while still giving me full type safety.

In `schema.js`, you can see the relational structure. The core table is `followups`, which holds the state, priority, and `dueAt` timestamps. I also created heavily indexed tables for `followup_events` to maintain an audit trail, and `notifications` to power the user's

inbox."

---

## 4. The Request Lifecycle (Routing, Middleware, Controllers)

---

*Explain how a standard API request flows through your app.*

### You say:

"To see how a standard request flows, let's look at the `modules` directory. I structured the codebase by feature module (e.g., Auth, Followups, Notifications) rather than grouping all controllers or all routes together.

When a request comes in, it hits `app.js`, is parsed, and then hits my global middleware. For example, any protected route goes through my `auth.middleware.js`. This middleware expects a JWT token—it first checks the `Authorization: Bearer` header, which I use for mobile and Safari, and falls back to checking `req.cookies` for desktop Chrome.

If validated, the request reaches the controller (e.g., `followup.controller.js`). The controllers handle the business logic, execute the Drizzle ORM queries, and return standardized JSON responses. Finally, if any asynchronous error occurs in the controller, it's caught by an `asyncHandler` wrapper and passed down to my global error handler middleware mounted at the very end of `app.js`."

---

## 5. The "Brain" of the App: The Reminder Engine

---

*This is where you impress them. Spend time here.*

## You say:

"The most complex part of the backend is the Reminder Engine, which lives in `jobs/reminder.job.js` and `services/reminder.service.js`.

I use `node-cron` to run a tick every 60 seconds. During that tick, the engine queries the database for any active follow-ups where the `dueAt` timestamp is in the past.

However, it doesn't just spam the user. It evaluates the user's selected policy (Normal, Persistent, or Aggressive) to calculate a cooldown window (60, 15, or 5 minutes). If the cooldown has expired, it fires a notification.

More importantly, it increments an `ignoreCount`. If that count crosses specific thresholds—say, the user ignores the task 3 times—the engine dynamically escalates the task's priority to High and pushes the notification severity to Urgent."

---

## 6. Real-Time Layer (Socket.IO)

---

*Explain how the engine talks to the frontend.*

## You say:

"When the Reminder Engine decides to fire a notification, it hooks into the Real-Time layer via `notification.service.js`.

I use Socket.IO for real-time WebSockets. When a user logs in, they establish a socket connection and join a private 'room' named after their `userId`.

So, when the backend cron job generates a new alert, the server simply calls `io.to(userId).emit('new_notification')`. This pushes the alert instantly to the React frontend.

I also configured Socket.IO with the **Redis Adapter**. I did this so that

if the backend scales to multiple instances behind a load balancer in the future, a cron job running on Server A can successfully push a WebSocket event to a user who happens to be connected to Server B."

---

## 7. Conclusion & Future Roadmap (End Here)

---

*Wrap up by showing you are forward-thinking.*

### You say:

"That's the core of the backend architecture: a standard Express REST API layered with a cron-based automation engine and real-time WebSockets.

If I were to scale this application further, the very first thing I would change is the cron architecture. Right now, the cron job executes synchronously in the Node process. To handle thousands of users, I would migrate that logic into a distributed job queue using **BullMQ**, which would allow a fleet of separate worker processes to process reminders in parallel without blocking the main API server."

---

### Quick Tips for Delivery:

---

1. **Don't rush:** Pause between sections. Ask "Does that make sense?" or "Would you like me to dive deeper into any of those modules?"
2. **Keep it conversational:** Don't read this script verbatim. Practice saying it out loud so it sounds like a natural explanation of code you wrote yourself.
3. **Control the screen:** If you are sharing your screen, physically open the files (`server.js`, `schema.js`, `reminder.job.js`) as you talk about them. Scroll to the relevant functions (like the graceful shutdown block) to anchor your explanation visually.